Fibonacci Series –

Space Complexity (Detailed Analysis)

Program:

```
// Fibonacci Series through Recursion

// This code prints the Fibonacci series using recursion.
The Fibonacci series is a series of numbers where the next
number is the sum of the previous two numbers. The first two
numbers of the series are 0 and 1. The series is as follows:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, etc.

#include <iostream>

using namespace std;

int fibonacci(int n) // Function to calculate the nth
Fibonacci number
{
    if (n == 0 || n == 1) // Base case
    {
        return n; // If n is 0 or 1, return n
    }
    return fibonacci(n - 1) + fibonacci(n - 2); // Recursive
call : nth Fibonacci number is the sum of (n-1)th and (n-
2)th Fibonacci number
}

int main() // Main function
{
    int n;                                  // Number of
terms
    cout << "Enter the number of terms: "; // Input the
number of terms
    cin >> n;                               // Store the
number of terms
```

```
    cout << "The Fibonacci series is: ";   // Output the
Fibonacci series
    for (int i = 0; i < n; i++)           // Loop to print
the first n Fibonacci numbers
    {
        cout << fibonacci(i) << " "; // Print the ith
Fibonacci number
    }
    return 0; // Return statement
}
```

Space Complexity :

1. ***Input Space***

Input space refers to the memory allocated for the variables and parameters provided to run the algorithm, independent of the algorithm's internal execution steps.

- ***Holistic Input Space***: *In* $main$ *function, allocated memory for an integer* n *(the user input) and a loop counter* i.
  Because these are simple primitive types (usually 4 bytes each in $C++$*), they take a constant amount of memory regardless of how large the input number grows. Therefore, the holistic input space is* $\boldsymbol{O}(\mathbf{1})$.

- ***Parametric Space***: *In the* $fibonacci(int\ n)$ *function, the parameter* n *is passed by value. Every time the function is called, memory is allocated to hold this integer within the activation record (stack frame) of that specific call.*
  Because it is a single integer, the parametric space per call is $\boldsymbol{O}(\mathbf{1})$.

3. Auxiliary Space (VERY IMPORTANT PART)

Now the real complexity comes from recursion.

Understanding recursion stack

Each call to:

```
    int n;
    cout << "Enter the number of terms: ";
    cin >> n;
    cout << "The Fibonacci series is: ";
    for (int i = 0; i < n; i++)
    {
        cout << fibonacci(i) << " ";
    }
```

$creates\ a\ stack\ frame\ containing$:

- $parameter\ n$
- $return\ address$
- $temporary\ storage$

$Each\ frame\ =\ \Theta(1) space\ per\ recursive\ call$

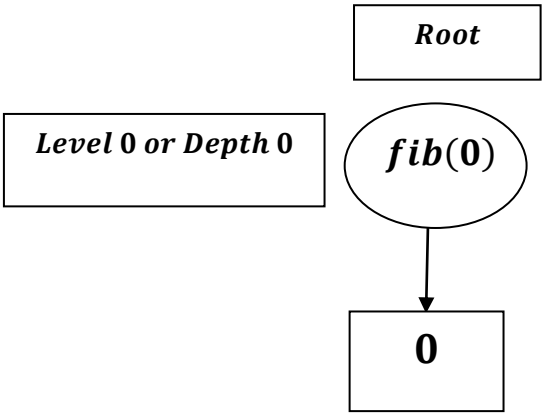
Maximum recursion depth

Even though Fibonacci creates ***two recursive calls****, they are*

not simultaneous in stack*.*

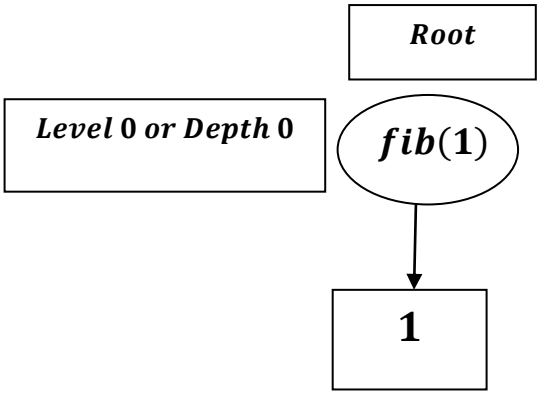
Execution is depth − first:

1) $\boldsymbol{When\ fibonacci(0)\ , we\ get\ a\ single\ node\ i.e.}$:



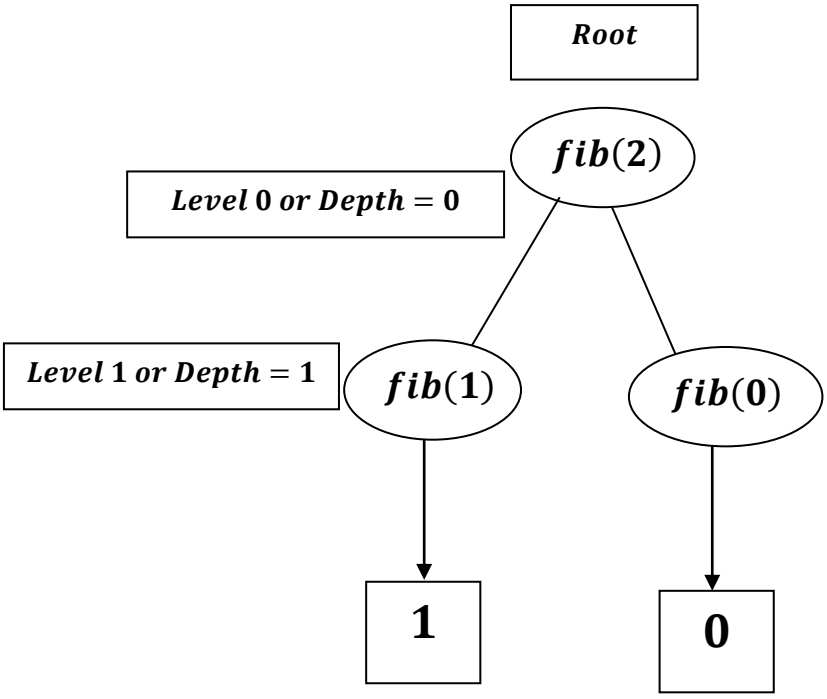
$$\boldsymbol{Nodes = Edges + 1(Root) = 0 + 1 = 1[Here\ there\ is\ no\ edges]}$$

2) $\boldsymbol{When\ fibonacci(1)\ , we\ get\ a\ single\ node\ i.e.}$:



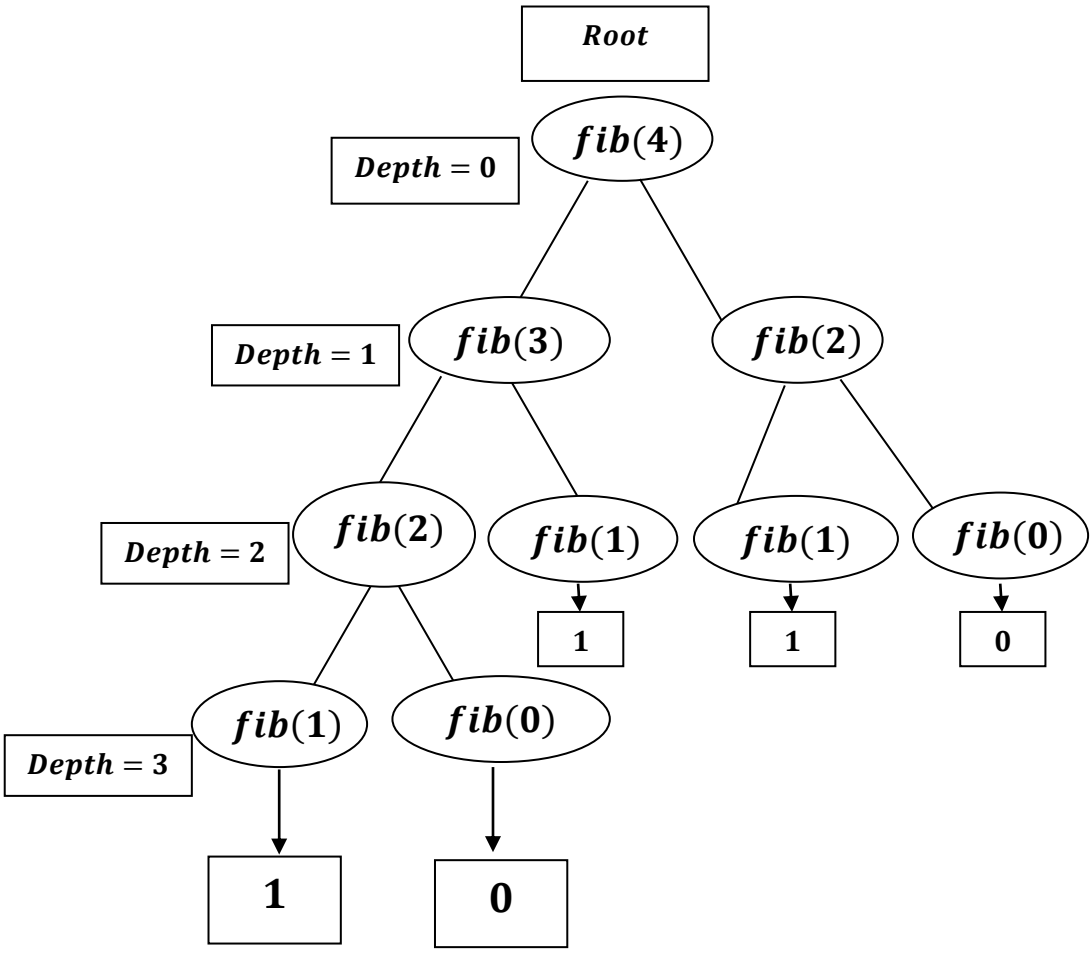
$$\boldsymbol{Nodes = Edges + 1(Root) = 0 + 1 = 1[Here\ there\ is\ no\ edges]}$$

$\mathbf{3)}\ \boldsymbol{When\ fibonacci(2)\ ,we\ get\ \ recursion\ tree}$:



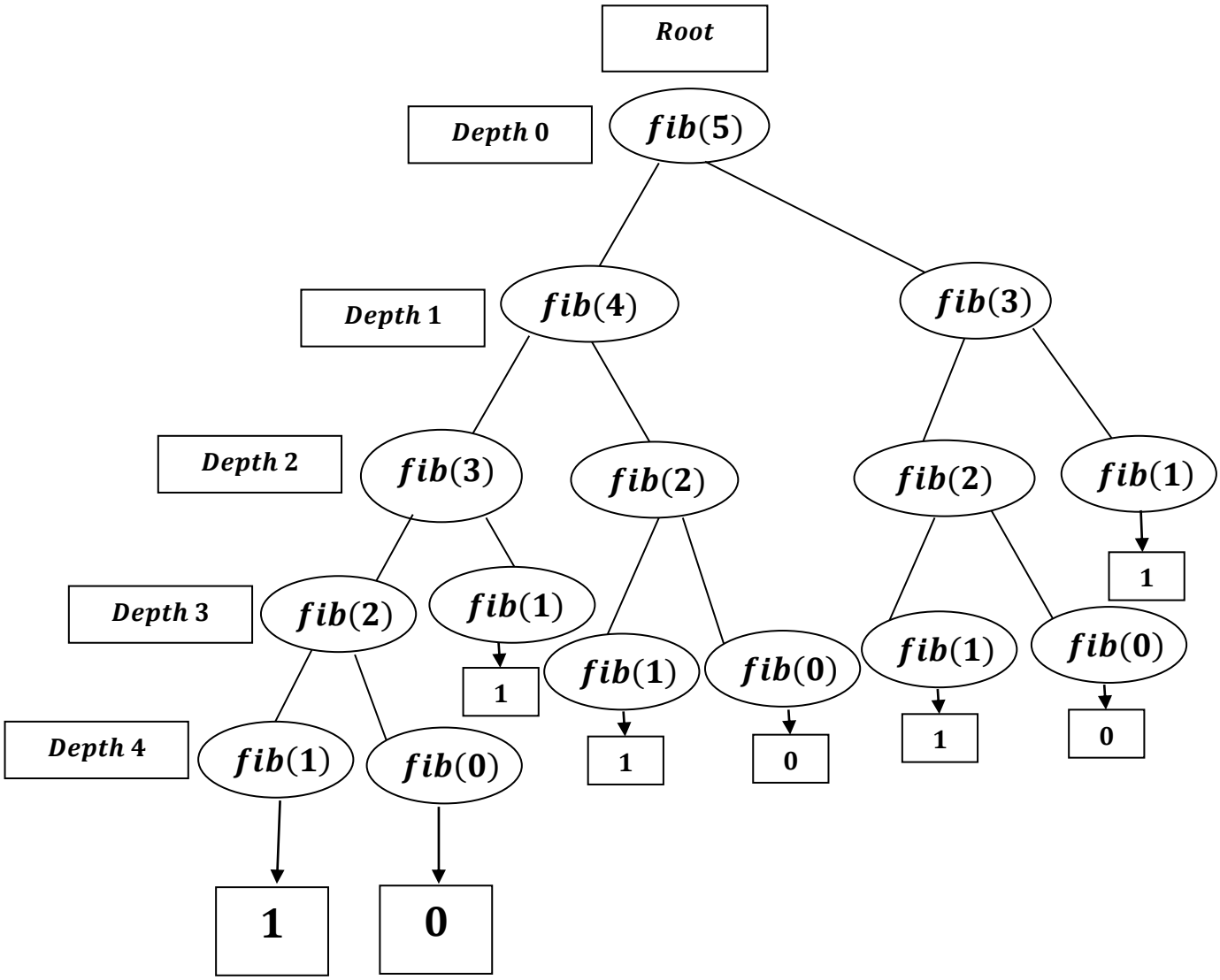
$$\boldsymbol{Nodes = Edges + 1(Root) = 1 + 1 = 2}$$

4) $When\ fibonacci(4)\ ,we\ get\ recursion\ tree$:



$$Nodes = Edges + 1(Root) = 3 + 1 = 4$$

$\boldsymbol{In\ Addition}$: $\boldsymbol{fibonacci(5)}$ $\boldsymbol{,we\ get\ recursion\ tree} \rightarrow$



$$\boldsymbol{Nodes = Edges + 1(Root) = 4 + 1 = 5}$$

$\boldsymbol{Now, for\ `n`\ \ , say\ stopping\ at\ fib(1)}.$

$\boldsymbol{fib(n) \rightarrow fib(n-1) \rightarrow \cdots \rightarrow fib(1)}$

$\boldsymbol{Then}$:

$\boldsymbol{n - k = 1[user\ input\ `n` - Edges(k) = 1]}$

$\boldsymbol{\Longrightarrow k = n - 1}$

$\boldsymbol{N(Nodes) = Edges(k) + 1(Root)}$

$\boldsymbol{\Longrightarrow N = (n-1) + 1 = n\ \ , then}$:

$\boldsymbol{Maximum\ Depth = Number\ of\ Edges = n - 1}.$

$\boldsymbol{Based\ on\ Active\ stackframe}$:

$$\therefore \boldsymbol{Total\ Auxiliary\ Space}$$
$$= (\boldsymbol{Maximum\ Depth} + \mathbf{1}) \times \begin{pmatrix} \boldsymbol{Space\ per} \\ \boldsymbol{Recursive\ Call} \end{pmatrix}$$

$$\begin{bmatrix} \boldsymbol{Here + 1\ for\ Root\ Node\ , which\ is\ also\ an\ active} \\ \boldsymbol{stack\ frame} \end{bmatrix}$$

$$= \boldsymbol{\Theta}(\mathbf{n} - \mathbf{1} + \mathbf{1}) \times \boldsymbol{\Theta}(\mathbf{1})$$
$$= \boldsymbol{\Theta}(\mathbf{n}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\boldsymbol{n})$$

$\boldsymbol{Now, for\ `n`\ \ , say\ stopping\ at\ fib(0)}.$

$\boldsymbol{fib(n) \rightarrow fib(n-1) \rightarrow \cdots \rightarrow fib(0)}$

$\boldsymbol{Then}$:

$\boldsymbol{n - k = 0[user\ input\ `n` - Edges(k) = 0]}$

$\boldsymbol{\Longrightarrow k = n}$

$\boldsymbol{N(Nodes) = Edges(k) + 1(Root)}$

$\boldsymbol{\Longrightarrow N = n + 1}$,

$\boldsymbol{Maximum\ Depth = Number\ of\ Edges = n}.$

$\boldsymbol{Based\ on\ Active\ stackframe}$:

$$\therefore \boldsymbol{Total\ Auxiliary\ Space}$$
$$= (\boldsymbol{Maximum\ Depth} + \mathbf{1}) \times \begin{pmatrix} \boldsymbol{Space\ per} \\ \boldsymbol{Recursive\ Call} \end{pmatrix}$$

$$\begin{bmatrix} \boldsymbol{Here} + \mathbf{1}\ \boldsymbol{for\ Root\ Node\ , which\ is\ also\ an\ active} \\ \boldsymbol{stack\ frame} \end{bmatrix}$$

$$= \boldsymbol{\Theta}(\mathbf{n} + \mathbf{1}) \times \boldsymbol{\Theta}(\mathbf{1})$$
$$= \boldsymbol{\Theta}(\mathbf{n}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\boldsymbol{n})$$

$\therefore$ $\boldsymbol{Total\ Space\ Complexity}$:

$$\boldsymbol{Input\ Space + Auxiliary\ Space = \Theta(1) + \Theta(n)}$$
$$= \boldsymbol{\Theta(n)}.$$